BOO BASH ARENA

Mario vs Pokémon | 16-bit Halloween turn-based multiplayer arcade

Complete implementation blueprint • Version 1.0 • 19 October 2026

Scope: playable 3v3 simultaneous-choice PvP, shared data and live analytics, three connected devices, 3–5 minute rounds.

Status: design specification and reference code/pseudocode; not a completed or play-tested software build. Numbers are initial balance assumptions requiring simulation and user testing.

1. Executive specification and acceptance criteria

• Two players join one room, choose opposite factions, draft exactly three distinct characters from five, arrange their lineup, and secretly choose one action each turn.

• A server resolves both committed actions simultaneously, animates the outcome, persists the event, and advances until one side loses three fighters or sudden death resolves the match.

• Two visitor devices are fully playable; a third device operates a live control/analytics dashboard. All devices communicate with the same backend.

• A completed match yields faction, character, composition and lead-order summaries, a validated win prediction, and a constrained draft recommendation.

• The game works on the local Wi-Fi network; support a new match after each result, reconnects, and clear failure handling.

2. Visual direction: authentic-feeling 16-bit pixel art

• Reference composition: opposing fighters in a side-view JRPG battle; upper-left opponent status, lower-right player status, lower-third command box; chunky outlined sprites.

• Target logical resolution: 320 × 180, integer-scaled to 1280 × 720 or 1600 × 900. Use nearest-neighbor scaling, disable antialiasing, and align UI to a 4-pixel grid.

• Halloween palette: midnight #241C36, purple #6943A8, pumpkin #F09A3E, eerie mint #A4E5A0, cream #F5E8D8, danger red #D65B63.

• Scene layers: moonlit sky, haunted castle silhouettes, drifting clouds, cemetery fence, grass/stone arena, character shadows, front sprites, HUD and battle log.

• Sprites: consistent 48 × 48 or 64 × 64 frames; idle 4 frames at 6 fps, attack 4–6 frames at 10 fps, hurt 2 frames, faint 4 frames, victory 4 frames.

• Audio: 8/16-bit-inspired loops, short attack SFX, separate mute controls; respect reduced-motion settings.

• Prepare original or appropriately licensed sprite/music assets. Fan use of Mario/Pokémon characters can carry intellectual-property restrictions; obtain permission if needed for public/commercial distribution.

3. Gameplay flow and screen inventory

Screen	Essential elements
Title	Start, rules, audio, credits, room join/create
Lobby	Room code, connected player states, faction selection, ready status
Draft	Five character cards, cost and stats, budget remaining, three slots, prediction/recommendation
Lineup	Drag/click reorder 1–3, lock lineup; hide choices until both lock
Battle	Sprites, HP/energy, bench portraits, turn counter, action buttons, secret locked indicator, battle log
Result	Winner, turns, damage, draft insight, rematch
Dashboard	Active rooms, session status, win rates, draft metrics, model validation, export CSV

Phase machine: LOBBY → FACTION_LOCK → DRAFT → LINEUP_LOCK → BATTLE → RESULT → REMATCH/LOBBY. Only the server may advance phases; reject messages that do not match current phase.

4. Initial character roster and role balance

Five characters per faction. Symmetric numerical stats by role; differentiated specials with equivalent intended power. Stats: HP, ATK, DEF, SPD, max energy 3. One active fighter at a time; two remain in bench slots.

Role	Mario side	Pokémon side	HP	ATK	DEF	SPD	Cost
Balanced	Mario	Charizard	40	10	5	5	4
Tank	Bowser	Snorlax	52	8	8	2	5
Speed	Luigi	Pikachu	32	9	3	9	4
Support	Peach	Blissey	36	7	5	5	4
Trickster	Boo	Gengar	34	9	4	7	4

Draft budget: 13 points for exactly 3 characters; with these costs every trio is legal except any trio containing Bowser/Snorlax plus two cost-5 units (none exist). Thus the current budget does NOT constrain actual choices: to make optimization meaningful, set budget to 12 (tank + two cost-4 = 13 and becomes unavailable) or revise costs to [4,5,3,3,4] with budget 12. Recommended implementation uses [4,5,3,3,4], budget 12. These are game design choices, not facts from the assignment.

5. Actions, effects, and deterministic combat rules

Action	Cost	Exact base effect
Attack	0 energy	Deal max(1, ATK + 4 - target DEF) physical damage; gain 1 energy (cap 3).
Defend	0 energy	Guard this turn: reduce incoming damage by 60% rounded up; gain 1 energy (cap 3).
Special	2 energy	Use character-specific ability defined below; no automatic energy gain. If insufficient energy, server rejects before lock.

• Each active character begins with 1 energy; energy persists when benched only if switching is added later (switching is excluded in v1).

• Simultaneous selection means the server waits for both valid locked commands or a 12-second timeout. On timeout, choose Defend automatically and log timeout.

• Guard is applied to all incoming damage on that turn. Effects use a fixed order: validate → spend energy → establish guards → resolve priority → calculate damage/effects → apply HP deltas together → determine faints → send events.

• For actions of equal priority, both effects are computed against start-of-turn state and applied simultaneously. Speed breaks only explicitly sequential special-effect ties, not ordinary attack simultaneity.

• All HP values clamp to [0, maxHP]; energy clamps to [0, 3]. No critical hits, misses, type advantages, or random damage in v1; this improves explainability and repeatability.

• Damage received while defending = ceil(rawDamage × 0.40). Healing never exceeds max HP. A fighter reduced to 0 HP still executes its already-locked move unless its special explicitly cancels; this supports true simultaneous resolution.

6. Complete named specials: initial balance design

Character	Special name	Cost	Effect
Mario	Fireball Burst	2	Deal 14 fixed fire damage; guard reduces it normally.
Charizard	Flame Burst	2	Deal 14 fixed fire damage; guard reduces it normally.
Bowser	Shell Slam	2	Deal 11 damage and gain 6 temporary shield HP, max 6.
Snorlax	Body Slam	2	Deal 11 damage and gain 6 temporary shield HP, max 6.
Luigi	Green Dash	2	Deal 11 damage; gain Evasion (negate next incoming attack or special damage this turn only, once).
Pikachu	Quick Volt	2	Deal 11 damage; gain Evasion (same rule).
Peach	Royal Remedy	2	Heal self for 13 HP; grant 3 shield HP (cap 6).
Blissey	Soft-Boiled	2	Heal self for 13 HP; grant 3 shield HP (cap 6).
Boo	Phantom Prank	2	Deal 8 damage; drain 1 enemy energy (floor 0); restore 1 own energy after paying.
Gengar	Shadow Trick	2	Deal 8 damage; drain 1 enemy energy (floor 0); restore 1 own energy after paying.

Resolution note: shields absorb damage before HP; shields persist until consumed, but reset when fighter faints. Evasion is set before simultaneous damage is applied; it negates one damage packet, not energy drain. For simultaneous drains, calculate from post-cost energy snapshots and apply together. A healing special can save a fighter because all heal and damage deltas are applied together.

7. Turn cap, knockouts, ties and sudden death

• A turn is one pair of resolved secret actions. After a faint, the next surviving fighter automatically enters at the start of the following turn. No manual swaps in MVP.

• Victory: a side has no living drafted fighters after all turn effects. Double elimination on the same turn is a draw, recorded separately; offer a rematch.

• At turn 20, if both sides survive, enter sudden death: each new turn inflicts 8 unavoidable damage to BOTH active fighters after action resolution, increasing by +4 each turn.

• Sudden death ends when a team is eliminated; hard stop at turn 25. At hard stop compare remaining fighter count, then total remaining HP fraction (sum HP/maxHP), then total damage dealt; exact tie is a draw.

• Timer: draft 30 s, lineup 10 s, each turn 12 s max. Worst-case theoretical duration exceeds five minutes, so test average session length; shorten to 8 s per turn and skip animations on timeout if needed.

8. Exact example turn and edge cases

Example: Mario (ATK 10, energy 2, HP 30) chooses Attack; Pikachu (DEF 3, energy 2, HP 22) chooses Special. Mario attack raw = 10 + 4 - 3 = 11. Pikachu Quick Volt deals 11 and activates Evasion, so Mario's 11 damage is negated. Mario loses 11 HP, gains 1 energy. Pikachu spends 2 energy, ends at 0. The result is computed from both locked actions.

• Both users submit twice: accept only first valid lock for the current turn, respond idempotently to duplicates.

• Disconnected player: preserve state and wait up to 20 s; then use automatic Defend each turn until reconnect, or forfeit after 60 s; make policy visible.

• Two simultaneous lethal blows: both execute; both faint; advance both benches; if neither has bench remaining, draw.

• Healer at full HP: heal amount is zero but shield still applies; action consumes energy.

• Special without enough energy: reject and prompt to choose Attack or Defend; timeout falls back to Defend.

• Server restart: store room snapshot and events; either restore unfinished rooms or mark abandoned, never silently award a win.

9. Recommended code architecture and folders

```
boo-bash-arena/
  client/                # Phaser + TypeScript
    src/scenes/          # Boot, Title, Lobby, Draft, Battle, Result
    src/ui/              # HPBar, ActionMenu, CharacterCard
    src/net/             # socket client + room state
    public/assets/       # licensed sprites, tiles, audio
  server/                # Node.js, TypeScript, Socket.IO
    src/game/            # roster, rules, reducer, combat, AI
    src/rooms/           # state machine, matchmaking
    src/analytics/       # features, predictors, optimizer
    src/db/              # schema, queries, migrations
  shared/                # types, event payload schemas
  scripts/               # simulation and balance testing
  docs/                  # diagrams, rules, validation
  tests/                 # unit, integration, load
```

Use one Node process for MVP with SQLite in WAL mode. All gameplay is authoritative on the server. The Phaser client renders received events and sends intentions only. Never let clients calculate or submit authoritative HP or winners.

10. Network protocol and security validation

Event	Direction	Minimum payload
room:create	client → server	playerName, clientVersion
room:join	client → server	roomCode, playerName
faction:select	client → server	roomId, faction
draft:lock	client → server	roomId, characterIds[3], orderedIds[3]
turn:lock	client → server	roomId, turnNumber, action, requestId
state:update	server → clients	phase, public state, currentTurn, serverTimestamp
turn:resolved	server → clients	turnNumber, action reveals, effects, HP, energy, faint, nextLead
match:finished	server → clients	winner, reason, summary
dashboard:update	server → dashboard	aggregate metrics, active room count

• Use 6-character random room codes with collision checks, 128-bit reconnect tokens, per-socket rate limits, JSON schema validation (Zod), and server-side faction/draft/action validation.

• Never broadcast a locked action until both are locked or timed out; send only an acknowledgement to the sender and a generic ready flag.

• Use monotonically increasing turn numbers and request IDs to prevent replay/duplicate commands.

• Expose dashboard in read-only mode to spectators; do not show secret action selections until turn resolution.

• Store minimal personal data: anonymous player IDs, optional nicknames, no passwords; explain collection and allow deleting exhibition data.

11. Data model: SQLite tables and relationships

```
CREATE TABLE matches (
 id TEXT PRIMARY KEY, room_code TEXT, started_at TEXT,
 ended_at TEXT, winner_faction TEXT, finish_reason TEXT,
 turns INTEGER, duration_seconds INTEGER, seed INTEGER,
 rules_version TEXT, model_version TEXT
);
CREATE TABLE players (
 id TEXT PRIMARY KEY, match_id TEXT NOT NULL,
 seat INTEGER, faction TEXT, anonymous_name TEXT,
 FOREIGN KEY(match_id) REFERENCES matches(id)
);
CREATE TABLE drafted_characters (
 match_id TEXT, player_id TEXT, character_id TEXT,
 lineup_position INTEGER, point_cost INTEGER,
 PRIMARY KEY(match_id,player_id,character_id)
);
CREATE TABLE turns (
 match_id TEXT, turn_no INTEGER, started_at TEXT,
 resolved_at TEXT, timeout_count INTEGER,
 PRIMARY KEY(match_id,turn_no)
);
CREATE TABLE actions (
 match_id TEXT, turn_no INTEGER, player_id TEXT,
 character_id TEXT, action_type TEXT, energy_before INTEGER,
 energy_after INTEGER, damage_dealt INTEGER, healing_done INTEGER,
 shield_added INTEGER, hp_before INTEGER, hp_after INTEGER,
 timed_out INTEGER, PRIMARY KEY(match_id,turn_no,player_id)
);
CREATE TABLE predictions (
 id TEXT PRIMARY KEY, match_id TEXT, phase TEXT,
 model_version TEXT, predicted_faction TEXT, probability REAL,
 feature_json TEXT, actual_outcome TEXT, created_at TEXT
);
CREATE TABLE recommendations (
 id TEXT PRIMARY KEY, match_id TEXT, player_id TEXT,
 roster_json TEXT, lineup_json TEXT, budget INTEGER,
 estimated_win_rate REAL, accepted INTEGER, model_version TEXT
);
```

Also create indexes on actions(match_id, player_id), matches(started_at), and predictions(model_version). Store events or replay snapshots for debugging; migrations should preserve schema versions. For production, use foreign keys and transactional inserts.

12. Data logging and metrics

• Log every draft, ordered lineup, turn choice, timeout, effect, HP/energy change, knockout, final outcome, and session duration with timestamps.

• Descriptive: total matches, Mario win %, Pokémon win %, draw %, average turns, median duration, role pick rate, character pick rate, composition win rate.

• Diagnostic: compare outcomes by first lead, role combinations, first three turns, timeout counts, and opponent role matchup; show sample sizes and avoid causal claims from observational data.

• Character win rate = wins among matches where selected / non-draw matches where selected. Composition win rate uses exact unordered trio; lead analysis treats order separately.

• Use Wilson 95% intervals for proportions; hide or label low-sample estimates, e.g. n < 20. Compare mirror roles and faction symmetry with simulation.

13. Predictive analytics: model, features and evaluation

Draft-stage model

Train logistic regression on simulated and then actual completed matches. Inputs: five binary role indicators for each side, point totals, lead role one-hot, faction flag, and cross-side role-pair indicators. Target: whether Mario won; exclude draws from binary training but report draw frequency. Use a seeded simulation and randomized legal draft strategies to avoid a biased dataset.

Mid-battle model

Inputs: turn number, alive count each side, normalized total HP remaining, active HP fractions, active energy, remaining shield, current active roles, and relative speed. Predict Mario win probability. Only use information known at the prediction time; do not leak future events.

Evaluation and safeguards

• Split by match ID, not turn rows. Use 70/15/15 train/validation/test with a fixed seed; keep a separate later real-game test set as it accumulates.

• Report test accuracy, Brier score, ROC AUC when classes permit, and calibration bins. Baseline = majority class and a simple HP advantage heuristic.

• Store model_version, training dataset size, split seed, training date, and observed evaluation metrics.

• Before sufficient data, use clearly labeled simulation-derived probability estimates, not invented accuracy. Retrain between exhibition sessions if enough new games arrive.

14. Prescriptive draft recommender: precise optimization

Decision: choose 3 distinct characters and order them (5P3 = 60 possibilities per faction), subject to total cost ≤ 12 using recommended costs [4,5,3,3,4]. Enumerate every legal ordered trio; evaluate each against the opponent's known faction and, if visible, their draft. Estimate expected win probability via 200–1000 seeded simulated matches per candidate, using a consistent opponent policy and random seed set for fair comparison. Rank feasible choices by estimated win rate; break ties by lower cost then alphabetical ID.

```
function recommend(roster, budget, opponentContext) {
  const choices = permutations(roster, 3)
    .filter(team => sum(team.map(x => x.cost)) <= budget);
  const ranked = choices.map(team => ({
    team, score: simulateWinRate(team, opponentContext,
                                 { trials: 500, seed: 20261027 })
  }));
  return ranked.sort((a,b) => b.score-a.score)[0];
}
```

Important: if opponent's lineup is hidden, optimize over a distribution of plausible opposing legal lineups rather than secretly reading it. Report the recommendation, cost, predicted chance, baseline alternative and simulation count. Show "insufficient evidence" if results are unstable. Recommender should never force player compliance.

15. Battle engine reference pseudocode

```
function resolveTurn(state, actionA, actionB) {
  assertValid(state, actionA, actionB);
  const snapshot = clone(state);
  const a = prepare(snapshot.A, actionA);
  const b = prepare(snapshot.B, actionB);
  // prepare spends energy and establishes guard/evasion
  const effectA = computeEffect(a, b, actionA);
  const effectB = computeEffect(b, a, actionB);
  // Apply both effects from same snapshot; shield before HP.
  applySimultaneous(state, effectA, effectB);
  grantAttackOrDefendEnergy(state, actionA, actionB);
  if (state.turn >= 20) applySuddenDeath(state);
  processFaintsAndBench(state);
  state.turn += 1;
  persistTurnTransaction(state, actionA, actionB);
  return buildPublicResolutionEvent(state);
}
```

Implementation detail: to avoid ambiguity, compute attack damage against the target’s start-of-turn DEF and guard; apply shield absorption using a single combined incoming damage packet per target. Damage, healing and shield effects are reconciled atomically. Explicitly test evasion/guard/sudden-death order.

16. Phaser scene and HUD implementation guide

• BootScene: load bitmap font, atlas, audio, tilemaps; enforce nearest-neighbor and responsive letterboxing.

• LobbyScene: create/join room, show six-character code and both connection states; handle reconnect token.

• DraftScene: display five animated portrait cards, selected trio, budget meter and recommendation panel; submit lock once.

• BattleScene: draw parallax haunted background, two sprites, HP bars, shield/energy icons, bench portraits, three large action buttons and scrolling battle text.

• Animations: animate each revealed action in a 0.6–1.2 second sequence, then reconcile to server state. Fast-forward and reduced-motion modes avoid slowing the match.

• Input: mouse/touch/keyboard keys 1/2/3, visible focus and selection, lock confirmation, disabled special when energy < 2.

• ResultScene: winner banner, performance summary, recommendation-versus-choice comparison, replay and rematch.

```
const game = new Phaser.Game({
 type: Phaser.AUTO, width: 320, height: 180,
 pixelArt: true, roundPixels: true,
 scale: { mode: Phaser.Scale.FIT,
          autoCenter: Phaser.Scale.CENTER_BOTH },
 scene: [BootScene, TitleScene, LobbyScene,
         DraftScene, BattleScene, ResultScene]
});
```

17. Dashboard layout and analytics API

Panel	Contents / endpoint
Live status	GET /api/rooms; connected users, active matches, last sync
Overview	GET /api/analytics/overview; counts, faction wins, durations
Characters	GET /api/analytics/characters; picks, wins, CI
Compositions	GET /api/analytics/compositions; trio and order comparisons
Predictions	GET /api/analytics/models; Brier score, calibration, test n
Export	GET /api/export/matches.csv and /api/export/actions.csv

Refresh dashboard via Socket.IO push after each resolved turn/match; throttle charts to once every 1–2 seconds. Mark simulated vs actual matches visibly and allow separate filters; do not blend them silently.

18. Testing matrix and balance simulation

Test group	Minimum assertions
Combat unit	Damage, guard rounding, shield absorption, heal cap, energy cap, drain floor, evasion, simultaneous lethal
Draft unit	3 distinct picks, correct side, valid lineup, budget rejection, all 60 permutations
Network integration	Secret choices, duplicate locks, stale turn, reconnect, timeout, cross-room isolation
Database	Match+turn atomic write, valid joins, migration, interrupted session recovery
Analytics	Known fixture counts, no train/test leakage, valid confidence intervals, CSV export
Exhibition	3 devices, 2 simultaneous player terminals, repeated rounds, 3–5 minute median

• Run at least 10,000 automated matches with seeded policies and randomized legal teams; inspect overall faction win rates, role matchups, draw rate, first-player advantage, median turns, and 95th-percentile duration.

• Balance target (design goal, not guarantee): Mario overall win rate 47–53% in non-draw simulated matches; no matched-role advantage beyond 5 percentage points without explanation.

• Because specials are symmetric by role, faction parity is expected under identical policies, but verify with paired seeds and swapped faction labels.

• Run 10–20 human pilot games, time draft and battle separately, gather usability issues, then adjust HP/damage or timers while incrementing rules_version.

19. Deployment and three-device operation

• Host the Node/Socket.IO server on the control laptop; connect all three laptops/tablets to the same reliable Wi-Fi/hotspot.

• Serve the Phaser web client at http://SERVER_LAN_IP:3000 and dashboard at /dashboard. Confirm firewall allows the port and devices can reach it.

• Control device shows live dashboard and manages rooms; player A and player B each use their own separate device.

• For the booth, print a QR code for the local URL, keep charging cables and spare hotspot ready, and have an offline local server setup.

• Use SQLite WAL and periodic backups; do not depend on external internet access. Test reconnect after Wi-Fi loss and page refresh.

• Use one shared deployment version on all devices and a version mismatch warning. Restart server only between matches when possible.

20. Eight-day implementation schedule (Oct 19–27)

Day	Deliverable
Oct 19	Finalize rules, roster, visual mockups, database schema, repository and tasks
Oct 20	Build combat reducer and comprehensive unit tests
Oct 21	Build Socket.IO rooms, secret action protocol, lobby and draft
Oct 22	Build Phaser battle HUD, action menu and simple animations
Oct 23	Add persistence, match results, dashboard and CSV export
Oct 24	Implement simulations, baseline prediction and constrained recommender
Oct 25	Balance tests, model evaluation, network and reconnection tests
Oct 26	Three-device full rehearsal, documentation, bug fixes and asset polish
Oct 27	Morning and afternoon arcade operation; collect logs and evidence

Scope guard: prioritize playable reliable multiplayer, recorded data, and defensible analytics over elaborate effects. If delayed, reduce animation variety rather than dropping three-device sync or prescriptive analytics.

21. Suggested group roles (4–5 members)

• Gameplay engineer: rules, reducer, tests, balance simulations.

• Frontend/pixel artist: Phaser scenes, sprite atlas, UI, animations and accessibility.

• Backend/network engineer: rooms, state machine, Socket.IO, SQLite, reconnect.

• Analytics engineer: data cleaning, summaries, predictor, evaluation, recommender.

• QA/booth lead (or shared): three-device rehearsals, documentation, deployment, exhibition and evidence.

22. Submission, demonstration and instructor checklist

• One shared Google Drive folder containing working source/build/deployment instructions, datasets, analytics scripts/models, brief documentation, and morning/afternoon session logs.

• Live demo: create room on two devices, pick opposing sides, draft and lock, resolve a turn with simultaneous secret actions, finish match, inspect stored records, show updated analytics and recommendation.

• Explain problem/objectives/KPIs/constraints (Chapter 2), data collection/validation (Chapter 3), descriptive/diagnostic comparisons (Chapter 4), predictive method and evaluation (Chapter 5), and constrained recommendation (Chapter 6).

• Clearly distinguish actual results from simulation, demonstrate at least one model evaluation, and explain why the recommendation is data-derived.

• Demonstrate replay, repeated matches, reconnect handling, and all three connected devices. No recorded video is required under the supplied brief.

23. Decisions to lock before coding

• Choose final five per faction: Mario, Bowser, Luigi, Peach, Boo vs Charizard, Snorlax, Pikachu, Blissey, Gengar.

• Use role costs [4,5,3,3,4] and 12-point budget; ensure legal draft combinations exist and recommender has meaningful constraints.

• Keep no manual swaps, no random critical hits, no elemental type chart, no hidden status stacks in v1.

• Use server-authoritative simultaneous actions and fixed damage numbers in this document as the canonical v1 rules.

• Use simulated data initially and collect real exhibition data; do not represent initial probabilities as proven predictions.

• Set a named rules_version and test all changes to stats, specials and sudden-death rules.

24. Source alignment and limitations

This specification was designed in response to the supplied nine-page “Pokémon Day III: Halloween Arcades” brief. That brief specifies the October 27, 2026 exhibition; creative freedom; genuine Chapters 1–6 analytics; acceptable synthetic/simulated/player data; three synchronized devices with two playable terminals; repeatable 3–5 minute sessions; functional descriptive, diagnostic, predictive and prescriptive analytics; and one Google Drive folder submission. It does not prescribe these characters, statistics, technologies, formulas or attacks: those are proposed project design decisions.

The attached visual reference is used only as direction for retro battle composition and pixel-art styling. This PDF does not contain a licensed sprite pack or claim to grant Mario/Pokémon intellectual-property rights. Implementers should verify library versions, compatibility, and assets when development begins.

END OF BLUEPRINT • Recommended first milestone: a headless combat engine with tests and two Socket.IO clients that can finish one match before adding pixel-art polish.